Recursion

2ip90

Kees Huizing

May a method call itself?

- Seems ridiculous

```
void f ( ) {
   System.out.println("Hello");
   f ( );
}
```

would never terminate and

print an infinite number of Hellos

```
class ThereIsAHoleInMyBucket {
  void getWater() {
    fixTheBucket();
  }

  void fixTheBucket() {
    putAStrawInTheHole();
  }

  void putAStrawInTheHole() {
    cutTheStraw();
  }

  void cutTheStraw() {
    grindTheAx();
  }

  void grindTheAx() {
    wetTheStone();
  }

  void wetTheStone() {
    getWater();
  }

  public static void main(String[] a) {
    new ThereIsAHoleInMyBucket()
      .getWater();
  }
}
```

May a method call itself?

- Seems ridiculous

```
void f ( ) {
   System.out.println("Hello");
   f ( );
}
```

would never terminate and

print an infinite number of Hellos

There's a Hole in My Bucket

```
class ThereIsAHoleInMyBu
  void getWater() {
    fixTheBucket();
  }

  void fixTheBucket() {
    putAStrawInTheHole();
  }

  void putAStrawInTheHole() {
    cutTheStraw();
  }

  void cutTheStraw() {
    grindTheAx();
  }

  void grindTheAx() {
    wetTheStone();
  }

  void wetTheStone() {
    getWater();
  }

  public static void main(String□ a) {
    new ThereIsAHoleInMyBucket()
      .getWater();
  }
}
```

with link to song
(skip this slide)

Nevertheless

- In Java, and many other programming languages, a method may call itself
- We call this *recursion* or sometimes *method recursion* (as opposed to *data recursion*, which refers to data that is structured in a recursive way)
- But why would we?

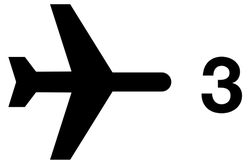

April, 1982
Ascension
6100 km
Falklands (Islas Malvinas)
© 2009 DMapas
© 2009 Europa Technologies
Data SIO, NOAA, U.S. Navy, NGA, GEBCO
© 2009 MapLink/Tele Atlas
29°46'11.80" S 27°19'07.50" W
©2009 Google
Eye alt 7090.50 km

The problem

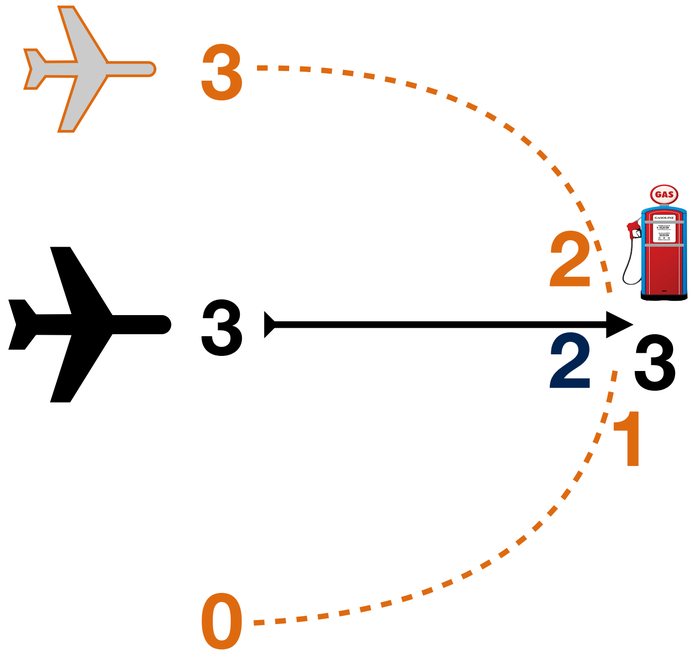
- one type of plane: 3000 km range (3 Mm for short)
- fuel can be used for flying or fueling another plane in mid-air
- fueling is considered instantaneous
- How many tanker flights do we need to bring one plane over a distance s, with full tanks?
- with the tanker planes safely returning to base.
- name f(s) is the function we want to investigate
  where s is the distance in Mm (thousands of km)

Small versions of the problem

- f(0) = 0

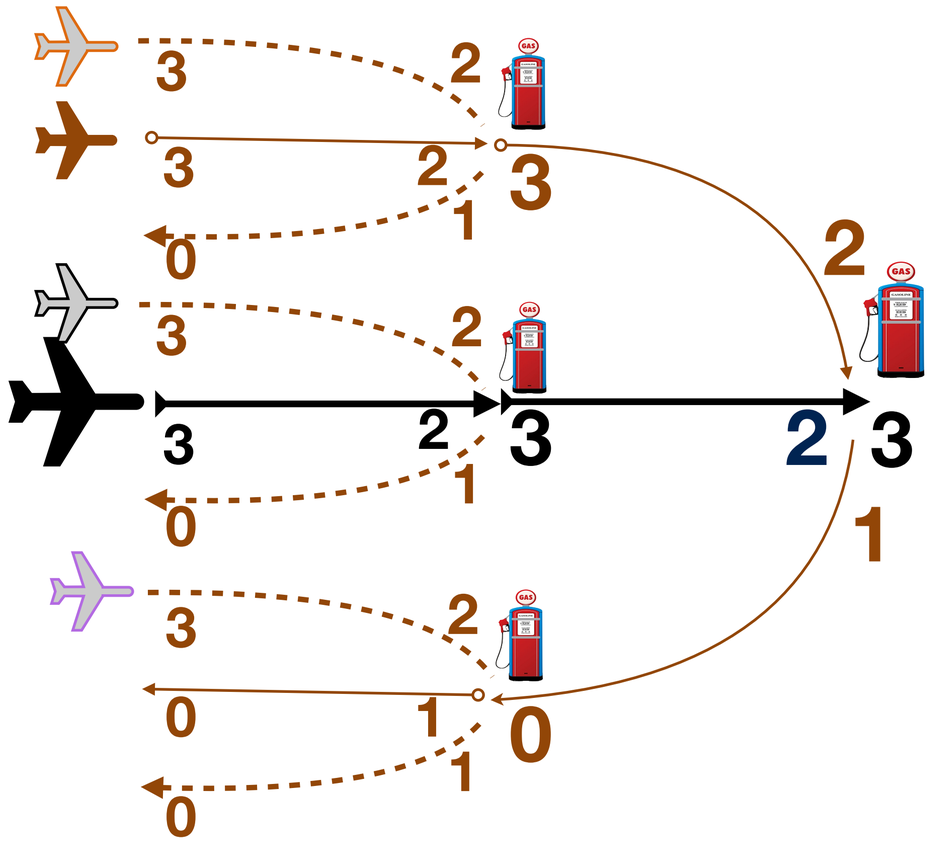


- f(1) = 1 (one tankerflight)



A slightly larger version

- f(2) = **1**
  - **+ 1**
  - **+ 1**
  - **+ 1**



etcetera...

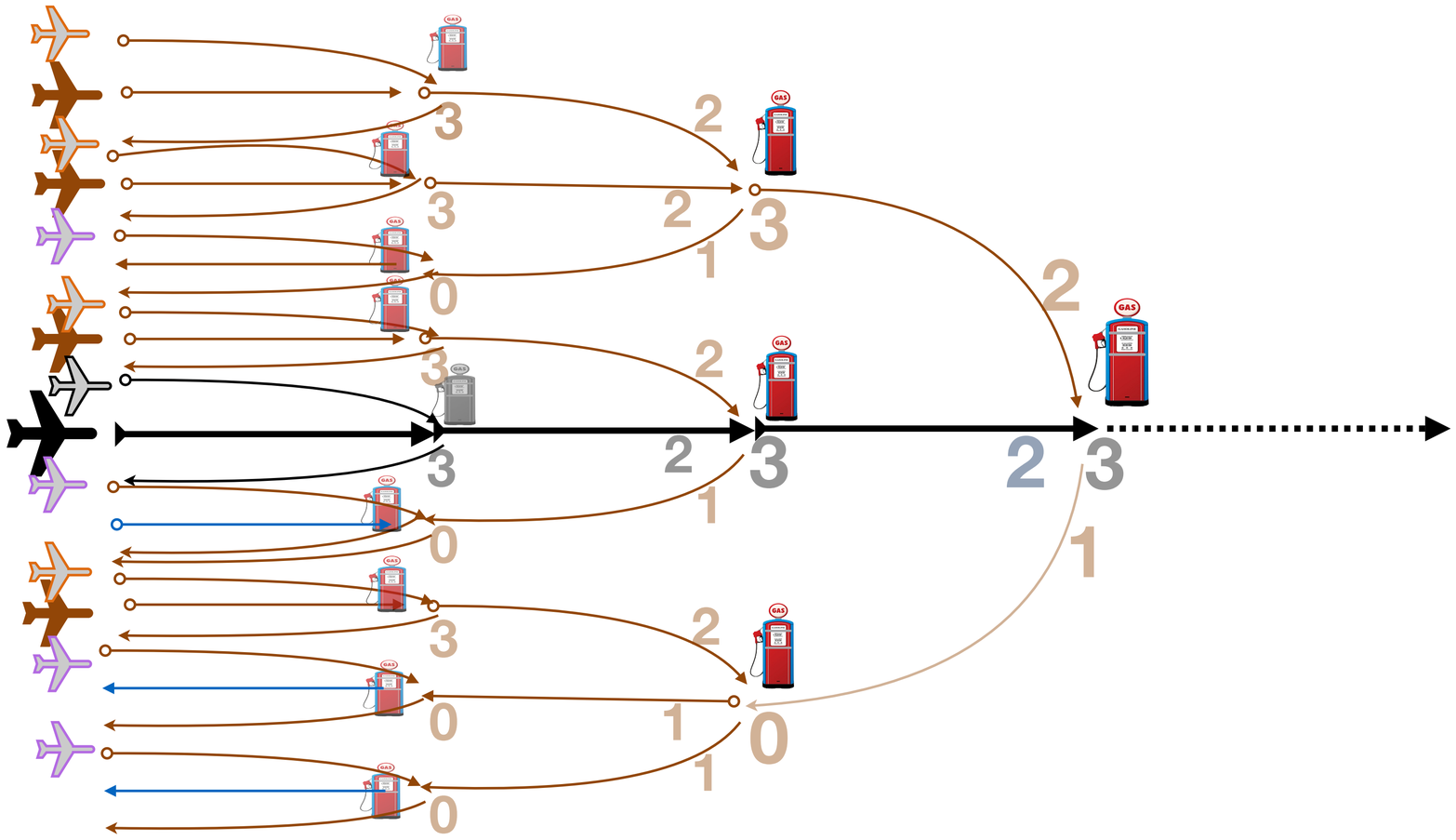
- f(3) =

previous **f(2)**

new forward + f(2)

new return + f(2)

new main + 1



Calculate f(s+1) from f(s)

- See blackboard

- f(0) = 0

- f(s+1) = f(s) +

f(s) +	the helpers we needed to get to s (including bringing them back)
1 +	the new helper plane H to bring us to s
f(s) +	the helpers to bring the new helper to s
f(s)	the helpers to bring the new helper back to base

= 3•f(s) + 1

Falklands - Quiz

- One tank load gives 3000 km (3 Mm)
- Tanker flight: 1000 km one way ⇒ deliver 1000 km worth ⇒1000 km return
- f(s): nr of tanker flights to support one plane flying *s Mm* starting and ending with tanks full




```
// @pre s >=0
// @post returns number of tanker flights needed to support
//          one airplane with full tanks to s Mm
int f(int s) {
    if (s == 0) {
        return 0;
    } else {
        return f(s-1)       // tankers for main plane s-1 Mm
               + 1          // support for main plane 1 Mm
               + f(s-1)     // support for tankers outward
               + f(s-1);    // support for tankers back
    }
}
```

Quiz how many

- How many tanker flights to deliver one plane with full tanks at 3000 km?

  Fill in a number



- One tank load gives 3000 km (3 Mm)
- Tanker flight: 1000 km one way ⇒ deliver 1000 km worth ⇒1000 km return
- f(s): nr of tanker flights to support one plane flying *s Mm* starting and ending with tanks full




```
// @pre s >=0
// @post returns number of tanker flights needed to support
//         one airplane with full tanks to s Mm
int f(int s) {
    if (s == 0) {
        return 0;
    } else {
        return f(s-1)       // tankers for main plane s-1 Mm
                + 1         // support for main plane 1 Mm
                + f(s-1)    // support for tankers outward
                + f(s-1);  // support for tankers back
    }
}
```


GAS
GASOLINE
THIS SALE
$ 0 0 0
GALLONS
0 0 0
PER GALLON

Recursion: general idea

- Divide problem into:
  - simple base case(s)
    - n=0 ⇒ return 1
  - reduction to *same* problem of *smaller* size
    - n>0 ⇒ n * fac(n-1)

- Often reduction to *several instances* of same problem
- Sometimes no direct self call, but via other method: f calls g, g calls f.
  This is called *mutual recursion*

(Method) Recursion

- typical situation: when a *recursive definition* or *recurrence relation* is known
- such as our tanker flights recurrence relation
- or the definition of the factorial function
  - n! = 1·2·3·…·n 3! = 6 5! = 120

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n((n-1)!) & \text{if } n > 0 \end{cases} \qquad \forall n \in \mathbb{N}.$$

(Wikipedia)

May a method call itself?

- Seems ridiculous

```
void f ( ) {
    System.out.println("Hello");
    f ( );
}
```

would never terminate print an infinite number of Hellos

- On the other hand… what if we would not *always* make a recursice call?

```
void g ( int n ) {
    if ( n == 0 ) {
        System.out.println( "." );
    } else {
        System.out.println( n );
        g( n-1 );
    }
}
```

g(4) will do:

print 4
call g(3), which will
print 3
call g(2), which will
print 2
call g(1), which will
print 1
call g(0), which will
print .

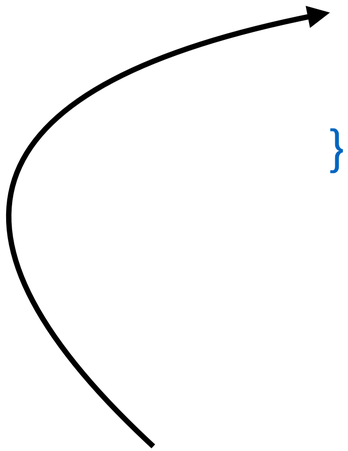
4
3
2
1
.

Quiz order



- What will this print?

```
void g (int n) {
    if (n == 0) {
        System.out.println( "." );
    } else {
        g(n-1);
        System.out.println(n);
    }
}
```



these two statements were switched

g(4) will print:

A
4
3
2
1
.

B
1
2
3
4
.

C
.
4
3
2
1

D
.
1
2
3
4

E
will not terminate

(Method) Recursion

- typical situation: when a *recursive definition* or *recurrence relation* is known
  - in this case: $n! = \begin{cases} 1 & \text{if } n = 0 \\ n((n-1)!) & \text{if } n > 0 \end{cases} \quad \forall n \in \mathbb{N}.$ (Wikipedia)
- the phenomenon that a method calls itself (possibly indirectly)

```
int fac(int n) {
   if (n==0) {
      return 1;
   } else {
      return n * fac(n-1);
   }
}
```

- note: long fac(int n) is better here, because a long can hold bigger numbers than an int; for the rest they are the same

Factorial

1. x = fac(3)
2. fac(3)
3. 3≠0, so calculate 3 * fac(2)
4. fac(2)
5. 2≠0, so calculate 2 * fac(1)
6. fac(1)
7. 1≠0, so calculate 1 * fac(0)
8. fac(0)
9. 0=0, so return 1 to step 8
10. return 1 * 1 to step 6
11. return 2 * 1 to step 4
12. return 3 * 2 to step 2
13. assign 6 to x

```
int fac(int n) {
    if (n==0) {
        return 1;
    } else {
        return n * fac(n-1);
    }
}
```

9a 7a 5a 3a 9b 7b 5b 3b 10 11 12 4 6 8

n in first call (step 2)	3
n in second call (step 4)	2
n in third call (step 6)	1
n in fourth call (step 8)	0

every recursive call creates a new set of parameters and local variables

at the call f(3), there are **four** variables (parameters) in memory that all have the name *n*

Quiz 1, 2, 3, 4



Let's check whether a String is a palindrome.

```
boolean isPalindrome(String s) {
    int len = s.length();

    if (   1   ) {                          // base case 1
        return true;
    } else if (                 2                ) { // base case 2
        return    3
    } else {                                // recursive case
        boolean result;
        result = isPalindrome(                 4                );
        return result;
    }
}
```

Note: s.substring(m, n) gives the substring of s from position m upto n

- What to fill in at **1**?

A. s.equals("")

B. len == 0

C. len == 1

D. len < 2

- What to fill in at **2**?

A. s.charAt(0) != s.charAt(len - 1)

B. vs.charAt(0) == s.charAt(len - 1)

C. s.charAt(0) != s.charAt(len)

D. as.charAt(0) == s.charAt(len)

- What to fill in at **3**?

A. 0

B. false

C. 1

D. true

- What to fill in at **4**?

.....

Quiz 1, 2, 3, 4

- What to fill in at **1**?

A. s.equals("")

Answer

Rabbits

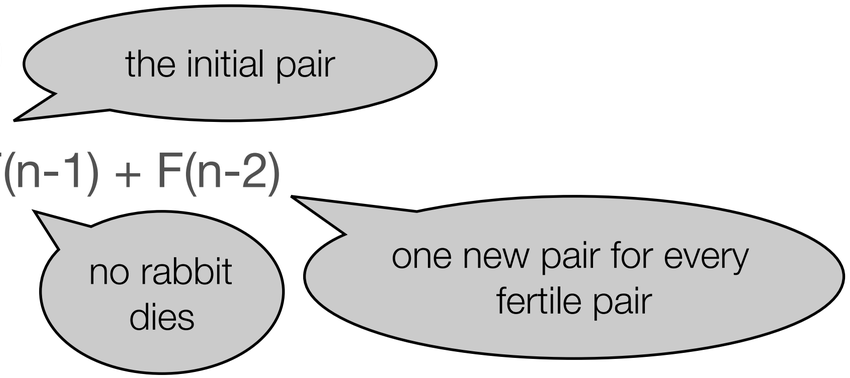
- The son of mr. Bonaccio gets a young pair of rabbits
- After one month, they are fertile and discover the flowers and the bees
- After one more month, mother gives birth to a healthy young pair of rabbits
  - after one month, they are fertile and discover the flower and the bees
  - after one more month, daughter gives birth to a healthy young pair of rabbits
    - after one month, they are fertile and discover the flower and the bees
    - after one more month, daughter gives birth to a healthy young pair of rabbits
- After one more month, mother gives birth to a healthy young pair of rabbits
  - ...

How many rabbits are there after *n* months?

Fibonacci

- Figlio di Bonnacio (Leonardo of Pisa, 1170-1250), mathematician

- F(0) = 0
  F(1) = 1
  F(n) = F(n-1) + F(n-2)



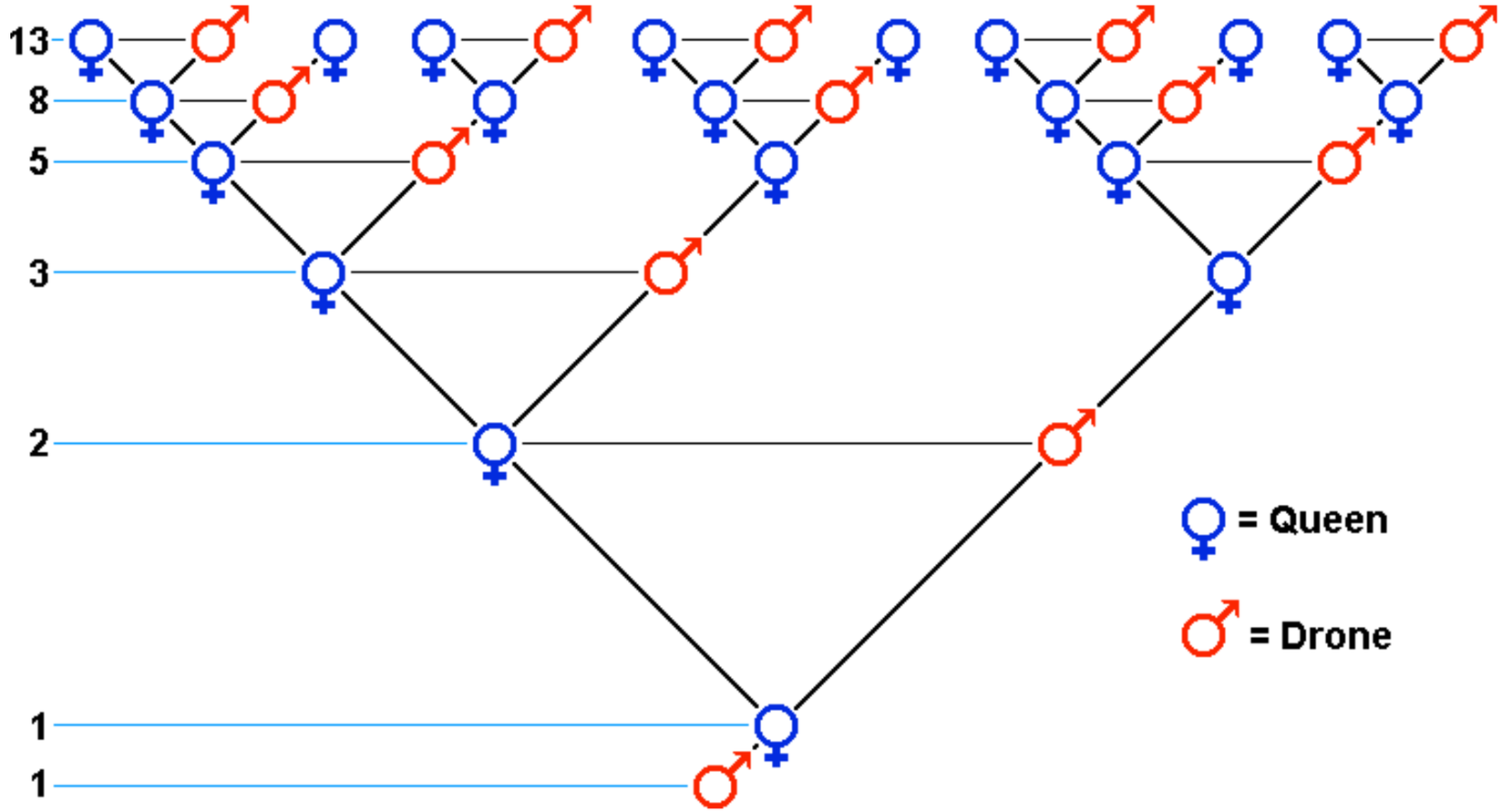
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

- a lot of interesting properties

  - F(n)/F(n-1) approaches (1+√5)/2, which is the *golden ratio* (gulden snede)

The genealogy of the drone

- A drone (male bee, *dar* in Dutch) is born from the queen without sex, i.e., he has no father
- A worker bee has a mother (queen) and a father (some drone)



Drones and workers (ctd.)

- Again these Fibonacci numbers! Can we understand this?

- 1 drone
  1 mother
  1 grandmother 1 grandfather
  2 grgrandmothers 1 grgrandfather
  3 grgrgrmothers 2 grgrgrfathers

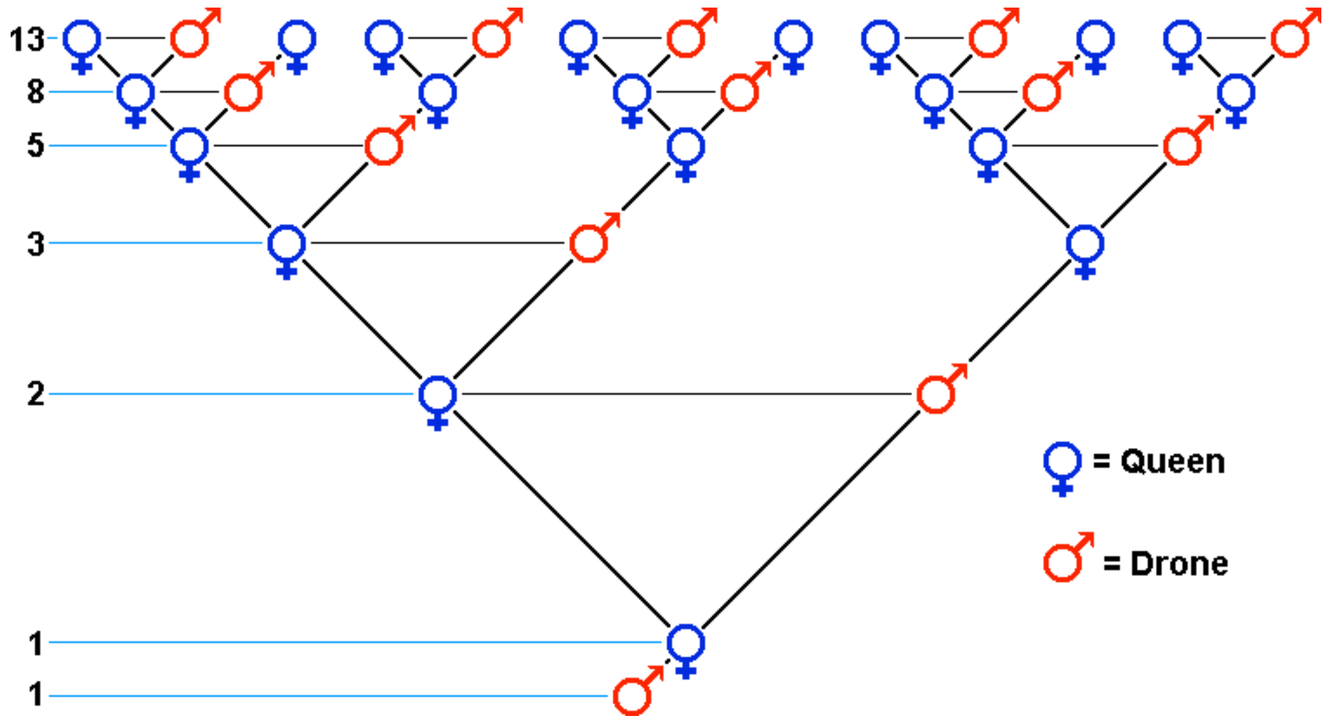
- d(n): drones in generation n
  q(n): queens in generation n

- f(n) = q(n) + d(n)

- d(n+1) = q(n)
  q(n+1) = q(n) + d(n)

- so q(n+1) = f(n)
  and d(n+1) = f(n-1)

- so f(n+1) = f(n) + f(n-1)



Recursive solution

```
//@pre n >= 0
//@post \returns nth Fibonacci number
int fibrec(int n) {
    if (n==0) {
        return 0;
    } else if (n==1) {
        return 1;
    } else { assert n > 1;
        return fibrec(n-1) + fibrec(n-2);
    }
}
```

- leads to many recursive calls
- e.g., fibrec(6) calls fibrec(2) 5 times, fibrec(2) 8 times, fibrec(0) 13 times !
- store intermediate results in global array (or HashMap) and take value instead of call ==> *Dynamic Programming*
- or find a dedicated *iterative* solution
- or find an efficient recursive method, using more parameters

Iterative solution

```
//@pre n >= 0
//@post \returns nth Fibonacci number
int fibit(int n) {
    int a = 0;
    int b = 1;
    int i = 0;

    while(i < n) {
        b = a + b;
        a = b - a;
        i++;
    }
    return a;
}
```

Iterative solution with annotation

```
//@pre n ≥ 0
//@post \returns Fib(n)
int fibit2(int n) {
    int a = 0;
    int b = 1;
    int i = 0;
    //@inv a == Fib(i) && b == Fib(i+1)
    while(i < n) {
        //assert a == A && b == B
        b = a + b;
        //assert a == A && b == A + B
        a = b - a;
        //assert a == B && b == A + B
        //assert a == Fib(i+1) && b == Fib(i+2)
        i++;
    }
    return a;
}
```

More examples of recursion

- Binomium: how many ways to choose *k* people from a group of *n*?
  - k = n? choose everyone, only 1 possibility
  - k = 0? leave everybody out, i.e., 1 possibility
  1. choose Isaac and then choose *k-1* people from the rest, a group of *n-1*
  2. don't choose Isaac, so choose *k* people from the rest, a group of *n-1*

- $$\binom{n}{n} = 1$$
  $$\binom{n}{0} = 1$$
  $$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad \text{if } n > k \geq 1$$

```
int binom(int n, int k) {
    if (n==k) {
        return 1;
    } else if (k==0) {
        return 1;
    } else {
        return binom(n-1, k-1) + binom(n-1, k)
    }
}
```

Dynamic storage needs

- problem: print lines of input in reverse order, last line first, etc.

- just 1 pass over input

  `aap noot mies` becomes `mies noot aap`

- Crab Canon (from: *Gödel, Escher, Bach* by Douglas R. Hofstadter, 1979)

- Problem: storage in an array is not easy, since number of lines is not known on beforehand. Suppose we cannot use ArrayList, how to do this?

Dynamic storage

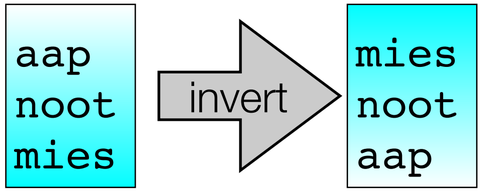
- Use recursion!

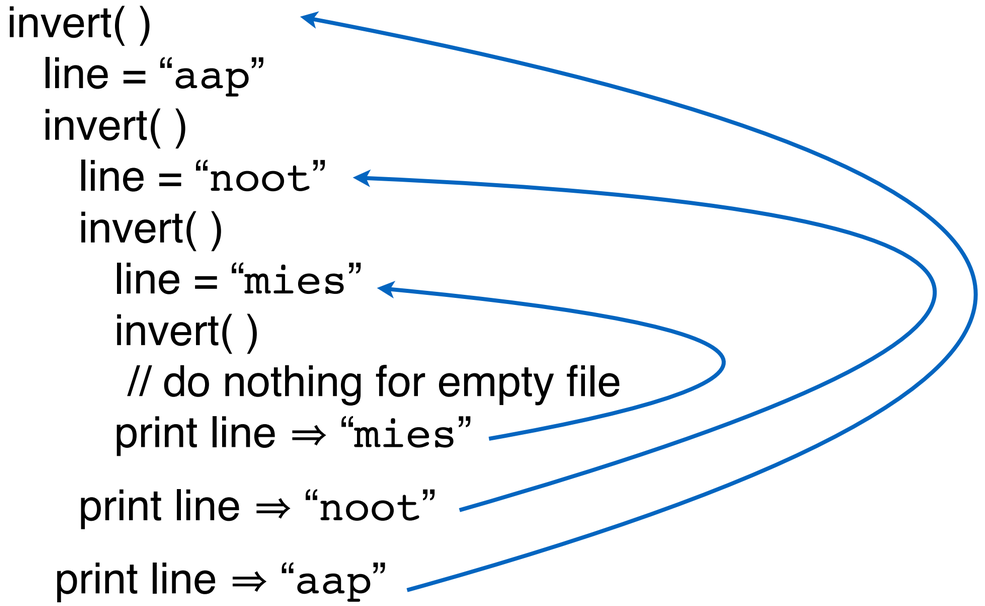
```
void invert() {
    if ( ! scanner.hasNext()) {
        // do nothing for empty file
    } else {
        String line;
        line = scanner.nextLine();
        invert();
        System.out.println( line );
    }
}
```

- every call of *invert* gives a new variable *line*

- there are other solut than recursion to use dynamic storage
  more about this later

Example





Recursion vs. iteration

- “opposite” of recursive: iterative

```
int fac_iterative(int n) {
      int f = 1;
      while (n>0) {
       f = n * f;
       n = n-1;
      }
      return f;
 }
```

recursive solution:

```
int fac(int n) {
   if (n==0) {
      return 1;
   } else {
      return n * fac(n-1);
   }
}
```

- the most efficient implementations are usually iterative

- recursive implementations are often shorter and **easier to write** (if the problem type is fit for recursion) and, in the end, easier to understand

Recursive design

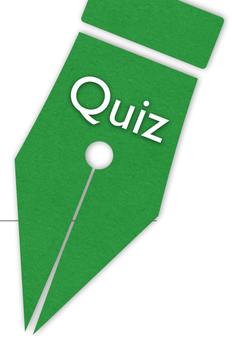
- Typical errors:
  - no or incomplete base cases
  - not always reduction in size

  } risk of no termination

  - too many steps in size reduction part: keep it as simple as possible!

- Formulate carefully what the recursive method solves then use this in the code that calls the method.

Quiz Recursion



- What is true about recursion? (multiple answers possible)

A. Recursion is the easiest way to program repetitive tasks

B. Only use recursion when there is no other way

C. Recursion is in general less efficient than iteration

D. Recursive programs tend to be longer than their iterative counterpart